

System2Module Forward Pass: Memory-Enhanced Reasoning Architecture

Mathematical Analysis

September 10, 2025

1 Notation Conventions

Index Notation: We use Einstein summation convention where repeated indices are summed over, except in Hadamard products where all indices must appear on both sides to avoid implicit summation.

Hadamard Product ($\odot$): Element-wise multiplication between tensors of identical dimensions. In index notation, all indices appear on both sides (e.g., $A_{ij} \odot B_{ij}$).

Broadcasting: When dimensions don't match for Hadamard product, we use broadcasting notation (e.g., $A_{ij}\alpha_i$ means α_i is repeated for each column j).

Softmax Representation: For any matrix $\mathbf{M}$, we define its softmax representation as $\mathbf{M}_S = \text{softmax}(\mathbf{M}, \text{dim} = -1)$.

Sigmoid Representation: For any matrix $\mathbf{M}$, we define its sigmoid representation as $\mathbf{M}_\sigma = \sigma(\mathbf{M})$ where $\sigma(x) = \frac{1}{1+e^{-x}}$.

Function Notation: We use $\text{RMSNorm}(\cdot)$ for RMS normalization, $\text{softmax}(\cdot)$ for softmax, and $\sigma(\cdot)$ for sigmoid consistently throughout.

2 Overview

The System2Module implements Kahneman's System 2 thinking through a sophisticated transformer architecture with persistent working memory. The forward pass combines input injection, multi-layer processing with memory enhancement, and reasoning history maintenance.

3 Mathematical Formulation

3.1 Input Processing

Given inputs:

$$\mathbf{H} \in \mathbb{R}^{B \times S \times D} \quad (\text{hidden states}) \quad (1)$$

$$\mathbf{I} \in \mathbb{R}^{B \times S \times D} \quad (\text{input injection}) \quad (2)$$

$$\mathbf{R} \in \mathbb{R}^{B \times P \times D} \quad (\text{reasoning history, optional}) \quad (3)$$

where B is batch size, S is sequence length, D is hidden dimension, and P is previous sequence length.

3.2 Input Injection

The first operation combines hidden states with input injection:

$$\mathbf{H}^{(0)} = \mathbf{H} + \mathbf{I} \in \mathbb{R}^{B \times S \times D} \quad (4)$$

Design Decision: Simple addition is used rather than gating. This choice assumes that input injection should be additive rather than selective, allowing the model to learn appropriate combinations through subsequent layers.

3.3 Layer-wise Processing

For each layer $l \in \{1, 2, \dots, L\}$ where L is the number of effective layers:

3.3.1 Working Memory Enhancement

The working memory mechanism operates on the current state $\mathbf{x} = \mathbf{H}^{(l-1)}$:

1. Memory Attention Computation:

$$\mathbf{A} = \mathbf{x}\mathbf{M}^T \in \mathbb{R}^{B \times S \times M} \quad (5)$$

where $\mathbf{M} \in \mathbb{R}^{M \times D}$ is the learnable memory buffer with M memory slots.

2. Attention Weight Normalization:

$$\mathbf{W}_S = \text{softmax}(\mathbf{A}, \text{dim} = -1) \in \mathbb{R}^{B \times S \times M} \quad (6)$$

3. Memory Retrieval:

$$\mathbf{R}_{\text{mem}} = \mathbf{W}_S \mathbf{M} \in \mathbb{R}^{B \times S \times D} \quad (7)$$

4. State Enhancement:

$$\mathbf{x}_{\text{enhanced}} = \mathbf{x} + \mathbf{R}_{\text{mem}} \in \mathbb{R}^{B \times S \times D} \quad (8)$$

Design Decision: Memory retrieval uses attention weights rather than gating. This allows for soft, weighted combination of memory slots rather than element-wise gating. The addition operation assumes memory should be additive rather than replacing current information.

3.3.2 Memory Retrieval Mechanisms

The working memory supports three retrieval strategies, selected via the `retrieval_strategy` parameter:

Strategy 1: Simple Retrieval When `retrieval_strategy = "simple"`:

Parameter Definitions:

- $\mathbf{M} \in \mathbb{R}^{M \times D}$: Memory matrix (learnable parameter)

1. Attention Computation:

$$\mathbf{A}_{\text{ret}} = \mathbf{x}\mathbf{M}^T \in \mathbb{R}^{B \times S \times M} \quad (9)$$

$$A_{\text{ret},bsi} = \sum_{j=1}^D x_{bsj} M_{ij} \quad (10)$$

2. Attention Weights:

$$\mathbf{W}_{\text{ret},S} = \text{softmax}(\mathbf{A}_{\text{ret}}, \text{dim} = -1) \in \mathbb{R}^{B \times S \times M} \quad (11)$$

$$W_{\text{ret},S,bsi} = \frac{\exp(A_{\text{ret},bsi})}{\sum_{i'=1}^M \exp(A_{\text{ret},bsi'})} \quad (12)$$

3. Memory Retrieval:

$$\mathbf{R}_{\text{mem}} = \mathbf{W}_{\text{ret},S} \mathbf{M} \in \mathbb{R}^{B \times S \times D} \quad (13)$$

$$R_{\text{mem},bsj} = \sum_{i=1}^M W_{\text{ret},S,bsi} M_{ij} \quad (14)$$

Asymptotic Complexity: $O(B \cdot S \cdot D \cdot M)$

- Attention computation: $O(B \cdot S \cdot D \cdot M)$
- Softmax computation: $O(B \cdot S \cdot M)$
- Memory retrieval: $O(B \cdot S \cdot M \cdot D)$

Memory Complexity: $O(B \cdot S \cdot M + M \cdot D)$

- Attention matrix: $O(B \cdot S \cdot M)$
- Memory matrix: $O(M \cdot D)$
- Intermediate tensors: $O(B \cdot S \cdot D)$

Strategy 2: Attention-Based Retrieval When `retrieval_strategy = "attention"`:

Parameter Definitions:

- $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{D \times D}$: Query, key, value projection matrices
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{D \times D}$: Output projection matrix
- $\mathbf{M} \in \mathbb{R}^{M \times D}$: Memory matrix (learnable parameter)

1. Multi-Head Attention:

$$\mathbf{R}_{\text{mem}}, \mathbf{W}_{\text{ret}, S} = \text{MultiHeadAttention}(\mathbf{x}, \mathbf{M}, \mathbf{M}) \in \mathbb{R}^{B \times S \times D} \times \mathbb{R}^{B \times S \times M} \quad (15)$$

$$\text{where: } \mathbf{Q} = \mathbf{x} \mathbf{W}_q, \quad \mathbf{K} = \mathbf{M} \mathbf{W}_k, \quad \mathbf{V} = \mathbf{M} \mathbf{W}_v \quad (16)$$

$$R_{\text{mem}, bsj} = \sum_{h=1}^H \sum_{i=1}^M \text{softmax} \left(\frac{\sum_{k=1}^D Q_{bsk} K_{ik}}{\sqrt{D/H}} \right) V_{ij} \quad (17)$$

$$W_{\text{ret}, S, bsi} = \text{softmax} \left(\frac{\sum_{k=1}^D Q_{bsk} K_{ik}}{\sqrt{D/H}} \right) \quad (18)$$

2. RMS Normalization:

$$\mathbf{R}_{\text{mem}} = \text{RMSNorm}(\mathbf{R}_{\text{mem}}) \in \mathbb{R}^{B \times S \times D} \quad (19)$$

$$R_{\text{mem}, bsj} = \frac{R_{\text{mem}, bsj}}{\sqrt{\frac{1}{D} \sum_{k=1}^D R_{\text{mem}, bsk}^2 + \epsilon}} \cdot w_j \quad (20)$$

where w_j is the learnable scale parameter and $\epsilon = 10^{-5}$ is a small constant for numerical stability.

Asymptotic Complexity: $O(B \cdot S \cdot D^2 + B \cdot S \cdot M \cdot D)$

- Attention computation: $O(B \cdot S \cdot D^2 + B \cdot S \cdot M \cdot D)$
- RMS normalization: $O(B \cdot S \cdot D)$

Memory Complexity: $O(B \cdot S \cdot M + M \cdot D + B \cdot S \cdot D)$

- Attention weights: $O(B \cdot S \cdot M)$
- Memory matrix: $O(M \cdot D)$
- Query/Key/Value projections: $O(B \cdot S \cdot D)$
- RMS normalization parameters: $O(D)$

Strategy 3: Sophisticated Retrieval When `retrieval_strategy = "sophisticated"`:

Parameter Definitions:

- $\alpha_{\text{ret}} \in \mathbb{R}^M$: Learnable memory decay factors for retrieval
- $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{D \times D}$: Query, key, value projection matrices
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{D \times D}$: Output projection matrix
- $\mathbf{W}_{\text{imp}} \in \mathbb{R}^{D \times 1}$: Learnable importance projection matrix
- $\mathbf{M} \in \mathbb{R}^{M \times D}$: Memory matrix (learnable parameter)

1. Memory Decay for Retrieval:

$$\mathbf{M}_{\text{decayed}} = \mathbf{M} \odot (\alpha_{\text{ret}} \mathbf{1}^T) \in \mathbb{R}^{M \times D} \quad (21)$$

$$M_{\text{decayed},ij} = M_{ij} \alpha_{\text{ret},i} \quad (\text{broadcasting: } \alpha_{\text{ret},i} \text{ repeated for each column } j) \quad (22)$$

2. Multi-Head Attention with Decayed Memory:

$$\mathbf{R}_{\text{mem}}, \mathbf{W}_{\text{ret},S} = \text{MultiHeadAttention}(\mathbf{x}, \mathbf{M}_{\text{decayed}}, \mathbf{M}_{\text{decayed}}) \in \mathbb{R}^{B \times S \times D} \times \mathbb{R}^{B \times S \times M} \quad (23)$$

$$\text{where: } \mathbf{Q} = \mathbf{x} \mathbf{W}_q, \quad \mathbf{K} = \mathbf{M}_{\text{decayed}} \mathbf{W}_k, \quad \mathbf{V} = \mathbf{M}_{\text{decayed}} \mathbf{W}_v \quad (24)$$

$$R_{\text{mem},bsj} = \sum_{h=1}^H \sum_{i=1}^M \text{softmax} \left(\frac{\sum_{k=1}^D Q_{bsk} K_{ik}}{\sqrt{D/H}} \right) V_{ij} \quad (25)$$

$$W_{\text{ret},S,bsi} = \text{softmax} \left(\frac{\sum_{k=1}^D Q_{bsk} K_{ik}}{\sqrt{D/H}} \right) \quad (26)$$

3. RMS Normalization:

$$\mathbf{R}_{\text{mem}} = \text{RMSNORM}(\mathbf{R}_{\text{mem}}) \in \mathbb{R}^{B \times S \times D} \quad (27)$$

$$R_{\text{mem},bsj} = \frac{R_{\text{mem},bsj}}{\sqrt{\frac{1}{D} \sum_{k=1}^D R_{\text{mem},bsk}^2 + \epsilon}} \cdot w_j \quad (28)$$

where w_j is the learnable scale parameter and $\epsilon = 10^{-5}$ is a small constant for numerical stability.

4. Importance Weighting:

$$\mathbf{I}_{\text{weights},\sigma} = \sigma(\mathbf{x} \mathbf{W}_{\text{imp}}) \in \mathbb{R}^{B \times S \times 1} \quad (29)$$

$$I_{\text{weights},\sigma,bs1} = \sigma \left(\sum_{j=1}^D x_{bsj} W_{\text{imp},j1} \right) \quad (30)$$

$$\mathbf{R}_{\text{mem}} = \mathbf{R}_{\text{mem}} \odot \mathbf{I}_{\text{weights},\sigma} \in \mathbb{R}^{B \times S \times D} \quad (31)$$

$$R_{\text{mem},bsj} = R_{\text{mem},bsj} I_{\text{weights},\sigma,bs1} \quad (\text{broadcasting: } I_{\text{weights},\sigma,bs1} \text{ repeated for each column } j) \quad (32)$$

Asymptotic Complexity: $O(B \cdot S \cdot D^2 + B \cdot S \cdot M \cdot D)$

- Memory decay: $O(M \cdot D)$
- Attention computation: $O(B \cdot S \cdot D^2 + B \cdot S \cdot M \cdot D)$
- RMS normalization: $O(B \cdot S \cdot D)$
- Importance weighting: $O(B \cdot S \cdot D)$

Memory Complexity: $O(B \cdot S \cdot M + M \cdot D + B \cdot S \cdot D)$

- Attention weights: $O(B \cdot S \cdot M)$
- Memory matrix: $O(M \cdot D)$
- Decayed memory: $O(M \cdot D)$
- Query/Key/Value projections: $O(B \cdot S \cdot D)$
- Importance weights: $O(B \cdot S)$
- RMS normalization parameters: $O(D)$

Design Decisions:

- **Strategy Selection:** Three levels of sophistication allow users to choose appropriate complexity for their task.
- **Memory Decay:** Sophisticated strategy includes separate decay factors for retrieval vs update.
- **Importance Weighting:** Sophisticated strategy weights retrieved memory based on current state importance.
- **Multi-Head Attention:** Attention and sophisticated strategies use multi-head attention for richer memory interactions.
- **RMS Normalization:** Used instead of LayerNorm for better efficiency and stability, following modern LLM practices (LLaMA, PaLM, etc.).

3.3.3 Memory Update Mechanisms

The working memory supports three update strategies, selected via the `update_strategy` parameter:

Strategy 1: Simple Update When `update_strategy = "simple"`:

Parameter Definitions:

- $\mathbf{W}_{\text{gate}} \in \mathbb{R}^{D \times M}$: Learnable gate projection matrix
- $\mathbf{W}_{\text{update}} \in \mathbb{R}^{2D \times D}$: Learnable update projection matrix
- $\mathbf{M} \in \mathbb{R}^{M \times D}$: Memory matrix (learnable parameter)

1. Representative Selection:

$$\mathbf{r} = \mathbf{x}[:, 0, :] \in \mathbb{R}^{B \times D} \quad (33)$$

$$r_{bj} = x_{b0j} \quad (\text{first token of each sequence}) \quad (34)$$

2. Update Gate Computation:

$$\mathbf{g}_\sigma = \sigma(\mathbf{r} \mathbf{W}_{\text{gate}}) \in \mathbb{R}^{B \times M} \quad (35)$$

$$g_{\sigma, bi} = \sigma \left(\sum_{j=1}^D r_{bj} W_{\text{gate}, ji} \right) \quad \text{where } \sigma(x) = \frac{1}{1 + e^{-x}} \quad (36)$$

3. Memory Update Preparation:

$$\bar{\mathbf{m}} = \frac{1}{M} \sum_{i=1}^M \mathbf{M}[i, :] \in \mathbb{R}^D \quad (37)$$

$$\bar{m}_j = \frac{1}{M} \sum_{i=1}^M M_{ij} \quad (38)$$

$$\mathbf{u} = \mathbf{W}_{\text{update}} \begin{bmatrix} \mathbf{r} \\ \bar{\mathbf{m}} \end{bmatrix} \in \mathbb{R}^{B \times D} \quad (39)$$

$$u_{bj} = \sum_{k=1}^{2D} W_{\text{update}, kj} \begin{bmatrix} r_{bk} \\ \bar{m}_k \end{bmatrix} \quad (40)$$

4. Vectorized Memory Update:

$$\bar{\mathbf{g}}_\sigma = \frac{1}{B} \sum_{b=1}^B \mathbf{g}_\sigma[b, :] \in \mathbb{R}^M \quad (41)$$

$$\bar{g}_{\sigma,i} = \frac{1}{B} \sum_{b=1}^B g_{\sigma,bi} \quad (42)$$

$$\bar{\mathbf{u}} = \frac{1}{B} \sum_{b=1}^B \mathbf{u}[b, :] \in \mathbb{R}^D \quad (43)$$

$$\bar{u}_j = \frac{1}{B} \sum_{b=1}^B u_{bj} \quad (44)$$

$$\mathbf{M} \leftarrow (1 - \bar{\mathbf{g}}_\sigma \mathbf{1}^T) \odot \mathbf{M} + (\bar{\mathbf{g}}_\sigma \mathbf{1}^T) \odot (\mathbf{1} \bar{\mathbf{u}}^T) \quad (45)$$

$$M_{ij} \leftarrow (1 - \bar{g}_{\sigma,i}) M_{ij} + \bar{g}_{\sigma,i} \bar{u}_j \quad (\text{broadcasting: } \bar{g}_{\sigma,i} \text{ and } \bar{u}_j \text{ create } M \times D \text{ matrices}) \quad (46)$$

Asymptotic Complexity: $O(B \cdot D \cdot M + B \cdot D^2)$

- Gate computation: $O(B \cdot D \cdot M)$
- Update preparation: $O(B \cdot D^2 + M \cdot D)$
- Memory update: $O(M \cdot D)$

Memory Complexity: $O(B \cdot M + M \cdot D + B \cdot D)$

- Gate matrix: $O(B \cdot M)$
- Memory matrix: $O(M \cdot D)$
- Update signal: $O(B \cdot D)$
- Gate projection matrix: $O(D \cdot M)$
- Update projection matrix: $O(2D \cdot D)$

Strategy 2: Attention-Based Update When `update_strategy = "attention"`:

Parameter Definitions:

- $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{D \times D}$: Query, key, value projection matrices
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{D \times D}$: Output projection matrix
- $\mathbf{M} \in \mathbb{R}^{M \times D}$: Memory matrix (learnable parameter)

1. Memory Attention:

$$\mathbf{A}_{\text{mem}}, \mathbf{W}_{\text{attn}, S} = \text{MultiHeadAttention}(\mathbf{x}, \mathbf{M}, \mathbf{M}) \in \mathbb{R}^{B \times S \times D} \times \mathbb{R}^{B \times S \times M} \quad (47)$$

$$\text{where: } \mathbf{Q} = \mathbf{x} \mathbf{W}_q, \quad \mathbf{K} = \mathbf{M} \mathbf{W}_k, \quad \mathbf{V} = \mathbf{M} \mathbf{W}_v \quad (48)$$

$$A_{\text{mem}, bsj} = \sum_{h=1}^H \sum_{i=1}^M \text{softmax} \left(\frac{\sum_{k=1}^D Q_{bsk} K_{ik}}{\sqrt{D/H}} \right) V_{ij} \quad (49)$$

$$W_{\text{attn}, S, bsi} = \text{softmax} \left(\frac{\sum_{k=1}^D Q_{bsk} K_{ik}}{\sqrt{D/H}} \right) \quad (50)$$

2. Update Signal:

$$\mathbf{s} = \frac{1}{S} \sum_{s=1}^S \mathbf{A}_{\text{mem}}[:, s, :] \in \mathbb{R}^{B \times D} \quad (51)$$

$$s_{bj} = \frac{1}{S} \sum_{s=1}^S A_{\text{mem}, bsj} \quad (52)$$

3. Attention-Based Gates:

$$\mathbf{g}_{\text{attn},\sigma} = \sigma \left(\frac{1}{S} \sum_{s=1}^S \mathbf{W}_{\text{attn},S}[:, s, :] \right) \in \mathbb{R}^{B \times M} \quad (53)$$

$$g_{\text{attn},\sigma,bi} = \sigma \left(\frac{1}{S} \sum_{s=1}^S W_{\text{attn},S,bsi} \right) \quad (54)$$

4. Vectorized Memory Update:

$$\bar{\mathbf{g}}_{\text{attn},\sigma} = \frac{1}{B} \sum_{b=1}^B \mathbf{g}_{\text{attn},\sigma}[b, :] \in \mathbb{R}^M \quad (55)$$

$$\bar{g}_{\text{attn},\sigma,i} = \frac{1}{B} \sum_{b=1}^B g_{\text{attn},\sigma,bi} \quad (56)$$

$$\bar{\mathbf{s}} = \frac{1}{B} \sum_{b=1}^B \mathbf{s}[b, :] \in \mathbb{R}^D \quad (57)$$

$$\bar{s}_j = \frac{1}{B} \sum_{b=1}^B s_{bj} \quad (58)$$

$$\mathbf{M} \leftarrow (1 - \bar{\mathbf{g}}_{\text{attn},\sigma} \mathbf{1}^T) \odot \mathbf{M} + (\bar{\mathbf{g}}_{\text{attn},\sigma} \mathbf{1}^T) \odot (\mathbf{1} \bar{\mathbf{s}}^T) \quad (59)$$

$$M_{ij} \leftarrow (1 - \bar{g}_{\text{attn},\sigma,i}) M_{ij} + \bar{g}_{\text{attn},\sigma,i} \bar{s}_j \quad (\text{broadcasting: } \bar{g}_{\text{attn},\sigma,i} \text{ and } \bar{s}_j \text{ create } M \times D \text{ matrices}) \quad (60)$$

Asymptotic Complexity: $O(B \cdot S \cdot D^2 + B \cdot S \cdot M \cdot D)$

- Attention computation: $O(B \cdot S \cdot D^2 + B \cdot S \cdot M \cdot D)$
- Update signal: $O(B \cdot S \cdot D)$
- Memory update: $O(M \cdot D)$

Memory Complexity: $O(B \cdot S \cdot M + M \cdot D + B \cdot S \cdot D)$

- Attention weights: $O(B \cdot S \cdot M)$
- Memory matrix: $O(M \cdot D)$
- Attention output: $O(B \cdot S \cdot D)$
- Query/Key/Value projections: $O(B \cdot S \cdot D)$
- Gate matrix: $O(B \cdot M)$

Strategy 3: Sophisticated Update When `update_strategy = "sophisticated"`:

Parameter Definitions:

- $\alpha \in \mathbb{R}^M$: Learnable memory decay factors
- $\mathbf{W}_{\text{imp}} \in \mathbb{R}^{D \times 1}$: Learnable importance projection matrix
- $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{D \times D}$: Query, key, value projection matrices
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{D \times D}$: Output projection matrix
- $\mathbf{W}_{\text{gate}} \in \mathbb{R}^{D \times M}$: Learnable gate projection matrix
- $\mathbf{M} \in \mathbb{R}^{M \times D}$: Memory matrix (learnable parameter)

1. Memory Decay:

$$\mathbf{M} \leftarrow \mathbf{M} \odot (\boldsymbol{\alpha} \mathbf{1}^T) \quad (61)$$

$$M_{ij} \leftarrow M_{ij} \alpha_i \quad (\text{broadcasting: } \alpha_i \text{ repeated for each column } j) \quad (62)$$

where $\mathbf{1} \in \mathbb{R}^D$ is a vector of ones, making $\boldsymbol{\alpha} \mathbf{1}^T \in \mathbb{R}^{M \times D}$ for proper Hadamard product dimensions.

2. Importance-Weighted Representative:

$$\mathbf{w}_S = \text{softmax}(\sigma(\mathbf{x} \mathbf{W}_{\text{imp}}), \dim = 1) \in \mathbb{R}^{B \times S} \quad (63)$$

$$w_{S,bs} = \frac{\exp(\sigma(\sum_{j=1}^D x_{bsj} W_{\text{imp},j1}))}{\sum_{s'=1}^S \exp(\sigma(\sum_{j=1}^D x_{bs'j} W_{\text{imp},j1}))} \quad (64)$$

$$\mathbf{r}_{\text{imp}} = \sum_{s=1}^S \mathbf{w}_S[:, s] \odot \mathbf{x}[:, s, :] \in \mathbb{R}^{B \times D} \quad (65)$$

$$r_{\text{imp},bj} = \sum_{s=1}^S w_{S,bs} x_{bsj} \quad (\text{broadcasting: } w_{S,bs} \text{ repeated for each column } j) \quad (66)$$

Role of $\mathbf{w}_S$ and $\mathbf{r}_{\text{imp}}$:

- **$\mathbf{w}_S$ (importance weights):** Softmax-normalized attention weights that determine how much each sequence position contributes to the representative. Each row sums to 1, creating a probability distribution over sequence positions.
- **$\mathbf{r}_{\text{imp}}$ (importance-weighted representative):** A single representative vector per batch item that combines all sequence positions according to their learned importance. This replaces the simple "first token" approach with a learned, context-aware selection mechanism.

3. Attention-Based Memory Interaction:

$$\mathbf{A}_{\text{soph}}, \mathbf{W}_{\text{soph},S} = \text{MultiHeadAttention}(\mathbf{r}_{\text{imp}}, \mathbf{M}, \mathbf{M}) \in \mathbb{R}^{B \times 1 \times D} \times \mathbb{R}^{B \times 1 \times M} \quad (67)$$

$$\text{where: } \mathbf{Q} = \mathbf{r}_{\text{imp}} \mathbf{W}_q, \quad \mathbf{K} = \mathbf{M} \mathbf{W}_k, \quad \mathbf{V} = \mathbf{M} \mathbf{W}_v \quad (68)$$

$$A_{\text{soph},b1j} = \sum_{h=1}^H \sum_{i=1}^M \text{softmax} \left(\frac{\sum_{k=1}^D Q_{b1k} K_{ik}}{\sqrt{D/H}} \right) V_{ij} \quad (69)$$

$$W_{\text{soph},S,b1i} = \text{softmax} \left(\frac{\sum_{k=1}^D Q_{b1k} K_{ik}}{\sqrt{D/H}} \right) \quad (70)$$

4. Adaptive Learning Rates:

$$\mathbf{g}_{\text{base},\sigma} = \sigma(\mathbf{r}_{\text{imp}} \mathbf{W}_{\text{gate}}) \in \mathbb{R}^{B \times M} \quad (71)$$

$$g_{\text{base},\sigma,bi} = \sigma \left(\sum_{j=1}^D r_{\text{imp},bj} W_{\text{gate},ji} \right) \quad (72)$$

$$\mathbf{g}_{\text{final}} = \mathbf{g}_{\text{base},\sigma} \odot \mathbf{W}_{\text{soph},S}[:, 0, :] \in \mathbb{R}^{B \times M} \quad (73)$$

$$g_{\text{final},bi} = g_{\text{base},\sigma,bi} W_{\text{soph},S,b0i} \quad (\text{true Hadamard product: all indices } b, i \text{ on both sides}) \quad (74)$$

$$\boldsymbol{\lambda} = \mathbf{g}_{\text{final}} \odot (\mathbf{1}(1 - \boldsymbol{\alpha})^T) \in \mathbb{R}^{B \times M} \quad (75)$$

$$\lambda_{bi} = g_{\text{final},bi}(1 - \alpha_i) \quad (\text{broadcasting: } \alpha_i \text{ repeated for each batch } b) \quad (76)$$

5. Vectorized Sophisticated Update:

$$\bar{\lambda} = \frac{1}{B} \sum_{b=1}^B \lambda[b, :] \in \mathbb{R}^M \quad (77)$$

$$\bar{\lambda}_i = \frac{1}{B} \sum_{b=1}^B \lambda_{bi} \quad (78)$$

$$\bar{\mathbf{u}}_{\text{soph}} = \frac{1}{B} \sum_{b=1}^B \mathbf{u}[b, :] \in \mathbb{R}^D \quad (79)$$

$$\bar{u}_{\text{soph},j} = \frac{1}{B} \sum_{b=1}^B u_{bj} \quad (80)$$

$$\mathbf{M} \leftarrow (1 - \bar{\lambda} \mathbf{1}^T) \odot \mathbf{M} + (\bar{\lambda} \mathbf{1}^T) \odot (\mathbf{1} \bar{\mathbf{u}}_{\text{soph}}^T) \quad (81)$$

$$M_{ij} \leftarrow (1 - \bar{\lambda}_i) M_{ij} + \bar{\lambda}_i \bar{u}_{\text{soph},j} \quad (\text{broadcasting: } \bar{\lambda}_i \text{ and } \bar{u}_{\text{soph},j} \text{ create } M \times D \text{ matrices}) \quad (82)$$

Asymptotic Complexity: $O(B \cdot S \cdot D + B \cdot D^2 + B \cdot M \cdot D)$

- Importance computation: $O(B \cdot S \cdot D)$
- Attention computation: $O(B \cdot D^2 + B \cdot M \cdot D)$
- Gate computation: $O(B \cdot D \cdot M)$
- Memory update: $O(M \cdot D)$

Memory Complexity: $O(B \cdot S + M \cdot D + B \cdot D + B \cdot M)$

- Importance weights: $O(B \cdot S)$
- Memory matrix: $O(M \cdot D)$
- Attention weights: $O(B \cdot M)$
- Query/Key/Value projections: $O(B \cdot D)$
- Gate matrices: $O(B \cdot M)$
- Decay factors: $O(M)$

Design Decisions:

- **Strategy Selection:** Three levels of sophistication allow users to choose appropriate complexity for their task.
- **Vectorization:** All strategies use batch-averaged updates for computational efficiency.
- **Representative Selection:** Simple uses first token, sophisticated uses importance-weighted average.
- **Memory Decay:** Only sophisticated strategy includes forgetting mechanisms.

3.3.4 Sophisticated Attention

The attention mechanism processes the memory-enhanced state:

1. Query, Key, Value Projections:

$$\mathbf{Q} = \mathbf{W}_q \mathbf{x}_{\text{enhanced}} \in \mathbb{R}^{B \times S \times D} \quad (83)$$

$$\mathbf{K} = \mathbf{W}_k \mathbf{x}_{\text{enhanced}} \in \mathbb{R}^{B \times S \times D} \quad (84)$$

$$\mathbf{V} = \mathbf{W}_v \mathbf{x}_{\text{enhanced}} \in \mathbb{R}^{B \times S \times D} \quad (85)$$

2. Multi-Head Reshaping:

$$\mathbf{Q} \in \mathbb{R}^{B \times H \times S \times D_h} \quad (86)$$

$$\mathbf{K} \in \mathbb{R}^{B \times H \times S \times D_h} \quad (87)$$

$$\mathbf{V} \in \mathbb{R}^{B \times H \times S \times D_h} \quad (88)$$

where H is the number of heads and $D_h = D/H$.

3. Rotary Position Embedding:

$$\mathbf{Q}' = \mathbf{Q} \odot \cos(\boldsymbol{\Theta}) + \text{rotate}(\mathbf{Q}) \odot \sin(\boldsymbol{\Theta}) \quad (89)$$

$$\mathbf{K}' = \mathbf{K} \odot \cos(\boldsymbol{\Theta}) + \text{rotate}(\mathbf{K}) \odot \sin(\boldsymbol{\Theta}) \quad (90)$$

where $\boldsymbol{\Theta} \in \mathbb{R}^{S \times D_h}$ contains position-dependent frequencies.

4. Scaled Dot-Product Attention:

$$\mathbf{A}_{\text{attn}, S} = \text{softmax} \left(\frac{\mathbf{Q}' \mathbf{K}'^T}{\sqrt{D_h}} \right) \in \mathbb{R}^{B \times H \times S \times S} \quad (91)$$

5. Attention Output:

$$\mathbf{O}_{\text{attn}} = \mathbf{A}_{\text{attn}, S} \mathbf{V} \in \mathbb{R}^{B \times H \times S \times D_h} \quad (92)$$

6. Reshape and Project:

$$\mathbf{O}_{\text{attn}} \in \mathbb{R}^{B \times S \times D} \rightarrow \mathbf{W}_{\text{out}} \mathbf{O}_{\text{attn}} \in \mathbb{R}^{B \times S \times D} \quad (93)$$

7. Reasoning History Attention (if available):

$$\mathbf{O}_{\text{hist}, -} = \text{MultiHeadAttention}(\mathbf{O}_{\text{attn}}, \mathbf{R}, \mathbf{R}) \in \mathbb{R}^{B \times S \times D} \quad (94)$$

8. Final Attention Output:

$$\mathbf{O}_{\text{final}} = \mathbf{O}_{\text{attn}} + \mathbf{O}_{\text{hist}} \in \mathbb{R}^{B \times S \times D} \quad (95)$$

Design Decision: Reasoning history is combined via addition rather than concatenation or gating. This maintains dimensional consistency while allowing the model to learn appropriate combinations through residual connections.

3.3.5 Residual Connection and Normalization

$$\mathbf{x}_{\text{attn}} = \mathbf{x}_{\text{enhanced}} + \mathbf{O}_{\text{final}} \in \mathbb{R}^{B \times S \times D} \quad (96)$$

$$\mathbf{x}_{\text{norm1}} = \text{RMSNorm}(\mathbf{x}_{\text{attn}}) \in \mathbb{R}^{B \times S \times D} \quad (97)$$

where RMSNorm is defined as:

$$\text{RMSNorm}(\mathbf{x}) = \mathbf{x} \odot \frac{1}{\sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2 + \epsilon}} \quad (98)$$

3.3.6 MLP Processing (SwiGLU)

1. Gate and Up Projections:

$$\mathbf{G} = \mathbf{W}_{\text{gate}} \mathbf{x}_{\text{norm1}} \in \mathbb{R}^{B \times S \times D_{\text{int}}} \quad (99)$$

$$\mathbf{U} = \mathbf{W}_{\text{up}} \mathbf{x}_{\text{norm1}} \in \mathbb{R}^{B \times S \times D_{\text{int}}} \quad (100)$$

2. Gated Activation:

$$\mathbf{O}_{\text{mlp}} = \mathbf{W}_{\text{down}} (\text{SiLU}(\mathbf{G}) \odot \mathbf{U}) \in \mathbb{R}^{B \times S \times D} \quad (101)$$

3. Residual Connection and Normalization:

$$\mathbf{x}_{\text{mlp}} = \mathbf{x}_{\text{norm1}} + \mathbf{O}_{\text{mlp}} \in \mathbb{R}^{B \times S \times D} \quad (102)$$

$$\mathbf{x}_{\text{final}} = \text{RMSNorm}(\mathbf{x}_{\text{mlp}}) \in \mathbb{R}^{B \times S \times D} \quad (103)$$

Design Decision: SwiGLU uses element-wise multiplication ($\odot$) rather than addition for gating. This allows for more sophisticated non-linear combinations and has shown superior performance in modern language models.

3.4 Reasoning History Update

After processing all layers, if in training mode:

$$\mathbf{R}_{\text{new}} = \begin{cases} \mathbf{H}^{(L)} & \text{if } \mathbf{R} = \text{None} \\ \text{concat}(\mathbf{R}, \mathbf{H}^{(L)})[:, -P_{\text{max}} :, :] & \text{otherwise} \end{cases} \quad (104)$$

where P_{max} is the maximum history length.

Design Decision: Concatenation is used for history accumulation rather than averaging or gating. This preserves the temporal sequence of reasoning steps, allowing the model to attend to specific historical positions.

4 Algorithm Summary

Algorithm 1 System2Module Forward Pass with Memory Update and Retrieval Strategies

```

1: Input:  $\mathbf{H} \in \mathbb{R}^{B \times S \times D}$ ,  $\mathbf{I} \in \mathbb{R}^{B \times S \times D}$ ,  $\mathbf{R} \in \mathbb{R}^{B \times P \times D}$ , update_strategy  $\in$ 
   {”simple”, ”attention”, ”sophisticated”}, retrieval_strategy  $\in$  {”simple”, ”attention”, ”sophisticated”}
2:  $\mathbf{H}^{(0)} \leftarrow \mathbf{H} + \mathbf{I}$ 
3: for  $l = 1$  to  $L$  do
4:    $\mathbf{x} \leftarrow \mathbf{H}^{(l-1)}$ 
5:    $\mathbf{x}_{\text{enhanced}} \leftarrow \text{WorkingMemory}(\mathbf{x}, \text{update\_strategy}, \text{retrieval\_strategy})$ 
6:    $\mathbf{x}_{\text{attn}} \leftarrow \mathbf{x}_{\text{enhanced}} + \text{Attention}(\mathbf{x}_{\text{enhanced}}, \mathbf{R})$ 
7:    $\mathbf{x}_{\text{norm1}} \leftarrow \text{RMSNorm}(\mathbf{x}_{\text{attn}})$ 
8:    $\mathbf{x}_{\text{mlp}} \leftarrow \mathbf{x}_{\text{norm1}} + \text{SwiGLU}(\mathbf{x}_{\text{norm1}})$ 
9:    $\mathbf{H}^{(l)} \leftarrow \text{RMSNorm}(\mathbf{x}_{\text{mlp}})$ 
10: end for
11: if training then
12:    $\mathbf{R} \leftarrow \text{UpdateHistory}(\mathbf{H}^{(L)})$ 
13: end if
14: Return:  $\mathbf{H}^{(L)}$ 

```

Algorithm 2 WorkingMemory with Vectorized Updates and Retrieval

```
1: Input:  $\mathbf{x} \in \mathbb{R}^{B \times S \times D}$ , update_strategy, retrieval_strategy, update_memory = True
2: if retrieval_strategy = "simple" then
3:    $\mathbf{A} \leftarrow \mathbf{xM}^T$  {Memory attention}
4:    $\mathbf{W}_S \leftarrow \text{softmax}(\mathbf{A}, \text{dim} = -1)$  {Attention weights}
5:    $\mathbf{R}_{\text{mem}} \leftarrow \mathbf{W}_S \mathbf{M}$  {Memory retrieval}
6: else if retrieval_strategy = "attention" then
7:    $\mathbf{R}_{\text{mem}}, \mathbf{W}_S \leftarrow \text{MultiHeadAttention}(\mathbf{x}, \mathbf{M}, \mathbf{M})$  {Multi-head attention retrieval}
8:    $\mathbf{R}_{\text{mem}} \leftarrow \text{RMSNorm}(\mathbf{R}_{\text{mem}})$  {RMS normalization}
9: else if retrieval_strategy = "sophisticated" then
10:   $\mathbf{M}_{\text{decayed}} \leftarrow \mathbf{M} \odot (\boldsymbol{\alpha}_{\text{ret}} \mathbf{1}^T)$  {Memory decay for retrieval}
11:   $\mathbf{R}_{\text{mem}}, \mathbf{W}_S \leftarrow \text{MultiHeadAttention}(\mathbf{x}, \mathbf{M}_{\text{decayed}}, \mathbf{M}_{\text{decayed}})$  {Multi-head attention with decayed mem-  
  ory}
12:   $\mathbf{R}_{\text{mem}} \leftarrow \text{RMSNorm}(\mathbf{R}_{\text{mem}})$  {RMS normalization}
13:   $\mathbf{I}_{\text{weights}, \sigma} \leftarrow \sigma(\mathbf{xW}_{\text{imp}})$  {Importance weights}
14:   $\mathbf{R}_{\text{mem}} \leftarrow \mathbf{R}_{\text{mem}} \odot \mathbf{I}_{\text{weights}, \sigma}$  {Importance weighting}
15: end if
16:  $\mathbf{x}_{\text{enhanced}} \leftarrow \mathbf{x} + \mathbf{R}_{\text{mem}}$  {State enhancement}
17: if update_memory then
18:   if update_strategy = "simple" then
19:      $\mathbf{r} \leftarrow \mathbf{x}[:, 0, :]$  {First token representative}
20:      $\mathbf{g}_{\sigma} \leftarrow \sigma(\mathbf{W}_{\text{gate}} \mathbf{r})$  {Update gates}
21:      $\mathbf{u} \leftarrow \mathbf{W}_{\text{update}}[\mathbf{r}; \mathbf{m}]$  {Update signal}
22:      $\mathbf{M} \leftarrow (1 - \bar{\mathbf{g}}_{\sigma} \mathbf{1}^T) \odot \mathbf{M} + (\bar{\mathbf{g}}_{\sigma} \mathbf{1}^T) \odot (\mathbf{1} \bar{\mathbf{u}}^T)$  {Vectorized update}
23:   else if update_strategy = "attention" then
24:      $(\mathbf{A}_{\text{mem}}, \mathbf{W}_{\text{attn}, S}) \leftarrow \text{MultiHeadAttention}(\mathbf{x}, \mathbf{M}, \mathbf{M})$ 
25:      $\mathbf{s} \leftarrow \frac{1}{S} \sum_{s=1}^S \mathbf{A}_{\text{mem}}[:, s, :]$  {Update signal}
26:      $\mathbf{g}_{\text{attn}, \sigma} \leftarrow \sigma(\frac{1}{S} \sum_{s=1}^S \mathbf{W}_{\text{attn}, S}[:, s, :])$  {Attention gates}
27:      $\mathbf{M} \leftarrow (1 - \bar{\mathbf{g}}_{\text{attn}, \sigma} \mathbf{1}^T) \odot \mathbf{M} + (\bar{\mathbf{g}}_{\text{attn}, \sigma} \mathbf{1}^T) \odot (\mathbf{1} \bar{\mathbf{s}}^T)$  {Vectorized update}
28:   else if update_strategy = "sophisticated" then
29:      $\mathbf{M} \leftarrow \mathbf{M} \odot (\boldsymbol{\alpha} \mathbf{1}^T)$  {Memory decay (row-wise scaling)}
30:      $\mathbf{w}_S \leftarrow \text{softmax}(\sigma(\mathbf{W}_{\text{imp}} \mathbf{x}), \text{dim} = 1)$  {Importance weights}
31:      $\mathbf{r}_{\text{imp}} \leftarrow \sum_{s=1}^S \mathbf{w}_S[:, s] \odot \mathbf{x}[:, s, :]$  {Importance representative (broadcasting:  $w_{S, bs}$  repeated for each  
  column)}
32:      $(\mathbf{A}_{\text{soph}}, \mathbf{W}_{\text{soph}, S}) \leftarrow \text{MultiHeadAttention}(\mathbf{r}_{\text{imp}}, \mathbf{M}, \mathbf{M})$ 
33:      $\boldsymbol{\lambda} \leftarrow \sigma(\mathbf{W}_{\text{gate}} \mathbf{r}_{\text{imp}}) \odot \mathbf{W}_{\text{soph}, S}[:, 0, :] \odot (\mathbf{1}(1 - \boldsymbol{\alpha})^T)$  {Adaptive learning rates}
34:      $\mathbf{M} \leftarrow (1 - \bar{\boldsymbol{\lambda}} \mathbf{1}^T) \odot \mathbf{M} + (\bar{\boldsymbol{\lambda}} \mathbf{1}^T) \odot (\mathbf{1} \bar{\mathbf{u}}^T)$  {Vectorized sophisticated update}
35:   end if
36: end if
37: Return:  $\mathbf{x}_{\text{enhanced}}$ 
```

5 Design Analysis

5.1 Combination Strategies

- **Addition vs. Concatenation:** Addition is used for residual connections and memory enhancement, maintaining dimensional consistency while allowing learned combinations. Concatenation is used for history accumulation to preserve temporal information.
- **Gating vs. Attention:** Memory retrieval uses attention for soft, weighted combinations. MLP processing uses gating (SwiGLU) for non-linear feature interactions.
- **Representative Selection:** First token selection prioritizes efficiency over information content, relying on the loss function to adapt the choice.

5.2 Memory Architecture

The working memory mechanism implements a dynamic memory system with three update strategies:

5.2.1 Strategy Comparison

Feature	Simple	Attention	Sophisticated
Representative Selection	First token	Attention-weighted	Importance-weighted
Memory Decay	No	No	Yes
Attention Mechanism	No	Yes	Yes
Adaptive Learning Rates	No	No	Yes
Computational Cost	Low	Medium	High
Memory Change Magnitude	High	Medium	Low

Table 1: Comparison of memory update strategies

5.2.2 Vectorization Benefits

All strategies use vectorized operations for computational efficiency:

- **Batch Averaging:** Update signals are averaged across batch dimension
- **Element-wise Operations:** Memory updates use broadcasting for efficiency
- **No Nested Loops:** Eliminated inefficient Python loops
- **GPU Acceleration:** Operations can run in parallel on GPU

5.2.3 Retrieval Strategy Comparison

Feature	Simple	Attention	Sophisticated
Attention Mechanism	Basic	Multi-head	Multi-head
Memory Decay	No	No	Yes
Importance Weighting	No	No	Yes
RMS Normalization	No	Yes	Yes
Computational Cost	Low	Medium	High
Retrieval Quality	Basic	Rich	Adaptive

Table 2: Comparison of memory retrieval strategies

5.2.4 Performance Characteristics

- **Simple Strategy:** Fastest execution, largest memory changes, minimal parameters
- **Attention Strategy:** Balanced performance, uses entire sequence information
- **Sophisticated Strategy:** Most controlled updates, includes forgetting mechanisms

5.2.5 Complexity Comparison

Strategy	Time Complexity	Memory Complexity	Dominant Factor
Simple Retrieval	$O(BSDM)$	$O(BSM + MD)$	Memory slots $\times$ dims
Attention Retrieval	$O(BSD^2 + BSMD)$	$O(BSM + MD + BSD)$	Attention computation
Sophisticated Retrieval	$O(BSD^2 + BSMD)$	$O(BSM + MD + BSD)$	Attention + importance
Simple Update	$O(BDM + BD^2)$	$O(BM + MD + BD)$	Gate computation
Attention Update	$O(BSD^2 + BSMD)$	$O(BSM + MD + BSD)$	Attention computation
Sophisticated Update	$O(BSD + BD^2 + BMD)$	$O(BS + MD + BD + BM)$	Importance + attention

Table 3: Computational and memory complexity comparison across all strategies

This design provides a flexible foundation for persistent memory with configurable sophistication levels while maintaining computational efficiency through vectorization.

6 Vectorization Improvements

6.1 Problem with Original Implementation

The original memory update implementation used nested loops:

```
for b in range(batch_size):
    for m in range(memory_size):
        memory[m] = (1 - gate[b,m]) * memory[m] + gate[b,m] * update[b]
```

This approach had several problems:

- **Poor Performance:** Loops are much slower than vectorized operations
- **No GPU Utilization:** Cannot take advantage of parallel processing
- **Memory Inefficient:** Multiple small operations instead of large vectorized ones
- **Not Scalable:** Performance degrades significantly with larger batch sizes

6.2 Vectorized Solution

The improved implementation uses batch-averaged vectorized operations:

$$\mathbf{M} \leftarrow (1 - \bar{\mathbf{g}}\mathbf{1}^T) \odot \mathbf{M} + (\bar{\mathbf{g}}\mathbf{1}^T) \odot (\mathbf{1}\bar{\mathbf{u}}^T) \quad (\text{broadcasting creates proper Hadamard dimensions}) \quad (105)$$

where $\bar{\mathbf{g}} = \frac{1}{B} \sum_{b=1}^B \mathbf{g}[b, :]$ and $\bar{\mathbf{u}} = \frac{1}{B} \sum_{b=1}^B \mathbf{u}[b, :]$.

Configuration	Simple	Attention	Sophisticated
Small (11064)	0.136 ms	0.617 ms	0.603 ms
Medium (42064)	0.329 ms	1.275 ms	1.477 ms
Large (850128)	0.534 ms	3.481 ms	2.999 ms
XLarge (16100256)	0.597 ms	11.352 ms	6.190 ms

Table 4: Memory update performance (milliseconds per update)

6.3 Performance Results

6.4 Key Benefits

- **Linear Scaling:** Performance scales much better with batch size
- **GPU Acceleration:** Operations can run in parallel on GPU
- **Memory Efficiency:** Single large operations instead of many small ones
- **Code Quality:** Cleaner, more maintainable vectorized code

A MultiHeadAttention Two-Array Return

This appendix explains why the `nn.MultiheadAttention` function returns two arrays and how they are used in the memory update strategies.

A.1 MultiHeadAttention Output Structure

The `MultiHeadAttention` function returns two distinct outputs:

$$\mathbf{A}_{\text{mem}}, \mathbf{W}_{\text{attn},S} = \text{MultiHeadAttention}(\mathbf{x}, \mathbf{M}, \mathbf{M}) \in \mathbb{R}^{B \times S \times D} \times \mathbb{R}^{B \times S \times M} \quad (106)$$

A.2 Output 1: Attention Output ($\mathbf{A}_{\text{mem}}$)

Purpose: Contains the memory-enhanced representation of the input.

Dimension: $\mathbb{R}^{B \times S \times D}$

What it represents: For each token in the input sequence $\mathbf{x}$, this contains a new representation that has integrated relevant information from memory $\mathbf{M}$.

Usage: This is the primary output that gets used as the update signal in the next step of the memory update process.

A.3 Output 2: Attention Weights ($\mathbf{W}_{\text{attn},S}$)

Purpose: Contains the attention scores showing how much each input token attends to each memory slot.

Dimension: $\mathbb{R}^{B \times S \times M}$

What it represents: For each batch item and sequence position, there's a vector of M attention scores (one per memory slot).

Usage: These weights are used to determine which memory slots to update and by how much.

A.4 Mathematical Breakdown

The `MultiHeadAttention(x, M, M)` operation works as follows:

- **Query:** $\mathbf{x}$ (current state) - "What am I looking for?"
- **Key:** $\mathbf{M}$ (memory buffer) - "What's available in memory?"

- **Value:** $\mathbf{M}$ (memory buffer) - "What information should I retrieve?"

Attention Computation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (107)$$

Where:

- $Q = \mathbf{x}$ (queries)
- $K = \mathbf{M}$ (keys)
- $V = \mathbf{M}$ (values)

A.5 Why Both Arrays Are Needed

A.5.1 For Memory Enhancement:

- $\mathbf{A}_{\text{mem}}$ provides the enhanced input representation
- This gets averaged across sequence length to create the update signal

A.5.2 For Memory Update:

- $\mathbf{W}_{\text{attn},S}$ provides the attention weights
- These weights determine which memory slots were most relevant
- Used to compute update gates: $\mathbf{g}_{\text{attn},\sigma} = \sigma(\text{mean}(\mathbf{W}_{\text{attn},S}))$

A.6 Usage in Memory Update Strategies

A.6.1 Attention-Based Strategy:

1. **Step 1:** Compute attention between input and memory

$$\mathbf{A}_{\text{mem}}, \mathbf{W}_{\text{attn},S} = \text{MultiHeadAttention}(\mathbf{x}, \mathbf{M}, \mathbf{M}) \quad (108)$$

2. **Step 2:** Use $\mathbf{A}_{\text{mem}}$ to create update signal

$$\mathbf{s} = \mathbf{A}_{\text{mem}} \cdot \text{mean}(\text{dim} = 1) \quad (\text{Average across sequence}) \quad (109)$$

3. **Step 3:** Use $\mathbf{W}_{\text{attn},S}$ to create update gates

$$\mathbf{g}_{\text{attn},\sigma} = \sigma(\mathbf{W}_{\text{attn},S} \cdot \text{mean}(\text{dim} = 1)) \quad (\text{Attention-based gates}) \quad (110)$$

4. **Step 4:** Apply vectorized update

$$\mathbf{M} \leftarrow (1 - \bar{\mathbf{g}}_{\text{attn},\sigma} \mathbf{1}^T) \odot \mathbf{M} + (\bar{\mathbf{g}}_{\text{attn},\sigma} \mathbf{1}^T) \odot (\mathbf{1} \mathbf{s}^T) \quad (\text{broadcasting creates proper Hadamard dimensions}) \quad (111)$$

where $\bar{\mathbf{g}}_{\text{attn},\sigma} = \frac{1}{B} \sum_{b=1}^B \mathbf{g}_{\text{attn},\sigma}[b, :]$ and $\bar{\mathbf{s}} = \frac{1}{B} \sum_{b=1}^B \mathbf{s}[b, :]$

A.6.2 Sophisticated Strategy:

The sophisticated strategy uses a similar pattern but with a single representative as the query:

1. **Step 1:** Compute attention between representative and memory

$$\mathbf{A}_{\text{soph}}, \mathbf{W}_{\text{soph},S} = \text{MultiHeadAttention}(\mathbf{r}_{\text{imp}}, \mathbf{M}, \mathbf{M}) \quad (112)$$

2. **Step 2:** Use both outputs for sophisticated memory updates with decay and importance weighting

A.7 Key Insight

The two outputs serve complementary roles:

- $\mathbf{A}_{\text{mem}}$: "What information was retrieved from memory?"
- $\mathbf{W}_{\text{attn},S}$: "How much attention was paid to each memory slot?"

This dual output allows the attention-based strategy to both enhance the current state with memory information AND use the attention patterns to intelligently update the memory buffer itself.

B Architectural Design vs. Learned Behavior

B.1 The Gap Between Design Intent and Training Reality

A critical consideration in the design of memory update mechanisms is the distinction between **what the architecture enables** and **what the model actually learns**. While the simple memory update strategy is designed to enable context-dependent learning through the formulation:

$$\mathbf{u} = \mathbf{W}_{\text{update}} \begin{bmatrix} \mathbf{r} \\ \bar{\mathbf{m}} \end{bmatrix} \quad (113)$$

the actual behavior depends entirely on the learned weights $\mathbf{W}_{\text{update}} \in \mathbb{R}^{D \times 2D}$.

B.2 The Hot Stove Analogy

Consider the analogy of learning about fire safety through touching a hot stove:

Baby's Experience:

- Limited existing knowledge about heat, pain, and safety
- Simple, direct association: "hot stove pain"
- Memory update: Basic safety rule formation

Teenager's Experience:

- Rich existing knowledge about chemistry, physics, and safety protocols
- Complex integration: "hot stove thermal energy transfer cellular damage pain response"
- Memory update: Sophisticated understanding of heat transfer mechanisms

Both receive the same sensory input (touching the hot stove), but their memory integration strategies differ dramatically based on their existing knowledge base $\bar{\mathbf{m}}$.

B.3 Theoretical vs. Practical Learning

Theoretical Capability: The architecture enables context-dependent updates through the concatenation $[\mathbf{r}, \bar{\mathbf{m}}]$, allowing the model to learn different integration strategies for different knowledge states.

Practical Reality: The learned weights $\mathbf{W}_{\text{update}}$ are determined by:

- **Loss function design:** What behavior is rewarded during training
- **Training data distribution:** What patterns the model observes
- **Optimization success:** Whether the model finds optimal solutions

B.4 Potential Learning Outcomes

The model might learn:

- $\mathbf{W}_{\text{update}} \approx [\mathbf{W}_r, \mathbf{0}]$: Ignores memory context entirely
- $\mathbf{W}_{\text{update}} \approx [\mathbf{0}, \mathbf{W}_m]$: Ignores new information entirely
- $\mathbf{W}_{\text{update}} \approx [\mathbf{W}_r, \mathbf{W}_m]$: Uses both components (intended behavior)

B.5 Empirical Validation Requirements

To verify that context-dependent learning actually occurs, empirical analysis must examine:

1. **Weight analysis:** Whether $\mathbf{W}_{\text{update}}$ utilizes both input components
2. **Behavioral testing:** Whether updates vary with memory state
3. **Performance comparison:** Whether context-dependent updates improve task performance
4. **Ablation studies:** Whether removing memory context degrades performance

B.6 Design Implications

This analysis highlights the importance of:

- **Careful loss function design** to encourage context-dependent behavior
- **Diverse training data** that demonstrates context-dependent patterns
- **Empirical validation** to verify that intended behaviors emerge
- **Architectural constraints** that make desired behaviors more likely to be learned

The memory update mechanism provides the *possibility* of sophisticated, context-dependent learning, but successful training is required to realize this potential.